

23B.1 — Weighted utilization mensual

DROP TABLE IF EXISTS

bu_frd.model_party_utilization_monthly_2022_2025;

CREATE TABLE

bu_frd.model_party_utilization_monthly_2022_2025

USING PARQUET

AS

SELECT

party_id,
observation_date,
feature_month,
feature_window,

/* -----
Eligible exposure
----- */

COUNT(
DISTINCT CASE
WHEN active_account_flag = 1
AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
IN ('C','R')
AND cr_lmt_amt > 0
THEN acct_id
END
) AS utilization_eligible_accounts,

SUM(
CASE
WHEN active_account_flag = 1
AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
IN ('C','R')
AND cr_lmt_amt > 0
THEN curr_bal_amt
END
) AS utilization_balance_sum,

SUM(
CASE
WHEN active_account_flag = 1
AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
IN ('C','R')
AND cr_lmt_amt > 0

```

        THEN cr_lmt_amt
    END
) AS utilization_limit_sum,

/* Current definition */
AVG(
    CASE
        WHEN active_account_flag = 1
            AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
                IN ('C','R')
            AND cr_lmt_amt > 0
        THEN
            1.0 * curr_bal_amt / cr_lmt_amt
        END
) AS avg_account_utilization,

/* New complementary definition */
CASE
    WHEN SUM(
        CASE
            WHEN active_account_flag = 1
                AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
                    IN ('C','R')
                AND cr_lmt_amt > 0
            THEN cr_lmt_amt
            ELSE 0
        END
    ) > 0

    THEN
        1.0 *
        SUM(
            CASE
                WHEN active_account_flag = 1
                    AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
                        IN ('C','R')
                    AND cr_lmt_amt > 0
                THEN curr_bal_amt
                ELSE 0
            END
        )
        /
        SUM(
            CASE
                WHEN active_account_flag = 1
                    AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
                        IN ('C','R')
                    AND cr_lmt_amt > 0

```

```

        THEN cr_lmt_amt
        ELSE 0
    END
)
ELSE NULL
END AS weighted_utilization,

/* Diagnostic */
MAX(
    CASE
        WHEN active_account_flag = 1
        AND UPPER(TRIM(CAST(porf_typ_cde AS STRING)))
            IN ('C','R')
        AND cr_lmt_amt > 0
        THEN
            1.0 * curr_bal_amt / cr_lmt_amt
        END
    ) AS max_account_utilization

FROM
    bu_frd.model_trades_transition_13m_2022_2025

WHERE feature_window IN ('PRIOR_9M','RECENT_3M')

GROUP BY
    party_id,
    observation_date,
    feature_month,
    feature_window;

```

Esta tabla debería tener las mismas:

8,263,068 party-months

Antes del Z-score: thresholds de TRAIN solamente

Quiero guardar la distribución de utilization exclusivamente en TRAIN:

```

SELECT

    percentile_approx(
        avg_utilization,
        ARRAY(

```

```

        0.95,
        0.99,
        0.995,
        0.999
    )
) AS avg_util_train_percentiles

FROM
    bu_frd.model_party_behavior_monthly_2022_2025

WHERE model_split = 'TRAIN'
    AND avg_utilization IS NOT NULL;

Y para weighted:

SELECT

    percentile_approx(
        weighted_utilization,
        ARRAY(
            0.95,
            0.99,
            0.995,
            0.999
        )
    ) AS weighted_util_train_percentiles,

    MIN(weighted_utilization) AS min_weighted_utilization,
    MAX(weighted_utilization) AS max_weighted_utilization

FROM
    bu_frd.model_party_utilization_monthly_2022_2025 u

INNER JOIN (
    SELECT DISTINCT
        party_id,
        observation_date,
        model_split
    FROM bu_frd.model_party_behavior_monthly_2022_2025
) c

ON u.party_id = c.party_id
    AND u.observation_date = c.observation_date

WHERE c.model_split = 'TRAIN'
    AND u.weighted_utilization IS NOT NULL;

```

Todavía no aplicamos ningún cap; solo guardamos los posibles thresholds.

Después sí pasamos a 23C

Y aquí hay una modificación importante respecto a lo que habíamos planeado: **23C no solo tendrá valores recent/prior; también tendrá coverage counts.**

Por ejemplo:

utilization_months_prior_9m
utilization_months_recent_3m

balance_months_prior_9m
balance_months_recent_3m

past_due_months_prior_9m
past_due_months_recent_3m

delinq_share_months_prior_9m
delinq_share_months_recent_3m

¿Por qué?

Porque estos dos casos no son equivalentes:

Party A:

recent utilization average = 0.70
available all 3 months

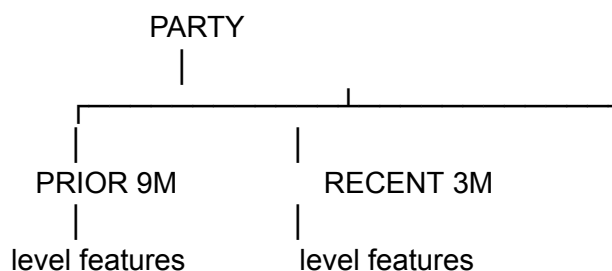
versus:

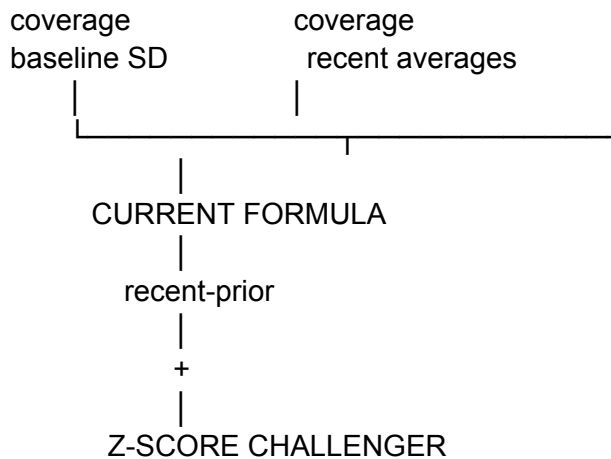
Party B:

recent utilization average = 0.70
available only 1 of 3 months

Queremos que el modelo conozca esa diferencia.

La estructura del 23C quedará así





Para utilizacion tendremos, por ejemplo:

avg_utilization_prior_9m
avg_utilization_recent_3m
utilization_change_vs_baseline

weighted_utilization_prior_9m
weighted_utilization_recent_3m
weighted_utilization_change

utilization_prior_months_available
utilization_recent_months_available

utilization_baseline_sd
utilization_zscore

Y lo mismo conceptualmente para las otras variables donde tenga sentido.

Mi lectura final de este QA

Los resultados nos dicen que:

1. La tabla party-month está estructuralmente correcta.
2. El missingness tiene significado económico/estructural, no parece ser únicamente un error de fuente.
3. CASES tienen mayor exposición crediticia activa de manera muy estable a través de TRAIN, VALIDATION y TEST.
4. Utilization tiene una cola extrema real pero muy pequeña.
5. No debemos convertir NULL a cero indiscriminadamente.

6. Vale la pena conservar average utilization y añadir weighted utilization.

Así que ahora haría solamente dos cosas antes del 23C: ejecutar el suplemento de **weighted utilization** y obtener los percentiles P95/P99/P99.5/P99.9 exclusivamente de TRAIN para avg_utilization y weighted_utilization.

Con esos resultados podremos escribir el **SQL completo de 23C una sola vez**, incluyendo la fórmula actual y la infraestructura necesaria para comparar después contra Z-score.